

PARALLAX

Backend, Security & Database Requirements

Implementation checklist for moving from prototype to production

Target stack: **Next.js 16** + **MySQL 8.4** (production & pre-production)

How to use this document

Every item is a checkbox. Work top to bottom — the phases are ordered by dependency, not by preference.

Tier 1 items block any real user. Tier 2 items block fest day. Tier 3 items are operational readiness.

☒ means done. ☐ with an amber note means the design is specified and verified but not yet applied to a running database — the schema exists on paper and in a test harness, not in production.

The five Go-Live Gates at the end are non-negotiable. Do not open registration until all five pass.

Revision note — this document was PostgreSQL, and is now MySQL

An earlier revision targeted PostgreSQL and Supabase. The database decision has changed to MySQL 8.4; the rationale, and the cost of that change, is recorded in DECISIONS.md.

Two capabilities the previous design relied on do not exist in MySQL and are not recovered by any translation: row-level security, and LISTEN/NOTIFY. Both are called out at the point where they matter rather than only here.

Everything else in the previous revision — the phasing, the payments analysis, the privacy obligations, the go-live gates — is unchanged and still applies.

1. Where the project stands

The application is feature-complete as a prototype. All screens, flows, and game mechanics work. What does not exist is a server: the entire data layer runs in the browser against localStorage.

1.1 What this means today

Area	Current state	Risk if shipped as-is
Accounts	localStorage; SHA-256 password hash in the browser	Anyone can read or forge any account
Roles	Client-side field	Any user can make themselves admin in devtools
Registrations	localStorage record	Editable; capacity limits bypassable
XP & achievements	Client-computed	Trivially inflated
Payment receipts	Base64 PDF in localStorage	Verification status is client-editable; will also exceed the ~5 MB quota

Area	Current state	Risk if shipped as-is
QR check-in	Not forgery-resistant	Attendance and prizes cannot be trusted
Data	Per-browser	Nothing syncs across devices; clearing the browser deletes the user

The single most important consequence

With raw MySQL credentials, the browser can never connect to the database directly — those credentials would be exposed to every visitor.

Every read and write must therefore move behind Server Actions or Route Handlers. That relocation, not the schema, is the bulk of the work.

1.2 What is already done and reusable

- The `Repository` interface (13 sub-repositories, 58 methods) is the contract a backend must satisfy. No screen imports a concrete implementation.
- Domain types mirror the intended column names one-to-one, so the mapping layer is a rename, not a redesign.
- All 27 tables are fully specified in `MYSQL-MIGRATION.md` — DDL, indexes, constraints, and both views — and the DDL has been applied to a live MySQL server with 25 invariant assertions passing.
- A 40-assertion acceptance suite exists at `/dev/data-test`. It must pass unchanged against the real backend.
- The four locked decisions are recorded in `DECISIONS.md`, including what each one costs.

2. Decisions to lock before writing code

These five choices constrain everything downstream. All are recorded with rationale in `DECISIONS.md`.

Decision	Choice	Why
Database	MySQL 8.4 (managed)	Project decision. Replaces the previous PostgreSQL/Supabase plan; see <code>DECISIONS.md</code> for the trade-offs accepted
Query layer	Drizzle ORM (<code>drizzle-orm/mysql2</code>)	Strong TypeScript inference, SQL-first migrations, maps cleanly onto the existing hand-written schema
Authentication	Auth.js (NextAuth v5) + Drizzle MySQL adapter	Lucia is no longer maintained; hand-rolled sessions are a poor trade for an app handling money
Authorization	App-layer checks + least-privilege split across two DB users	MySQL has no row-level security , so there is no per-row backstop. Grants and an audit log are the closest available substitute
Integrity	Constraints in the schema; sequencing in TypeScript transactions	MySQL has no <code>RETURNING</code> and a weak procedural story; <code>SELECT ... FOR UPDATE</code> replaces advisory locks correctly

Decision	Choice	Why
Payments	Payment gateway with signed webhook	Cryptographic proof of payment instead of a human eyeballing an editable PDF

2.1 Open question — hosting target

Under MySQL this no longer changes the realtime answer — announcements are polled on any host, because there is no LISTEN/NOTIFY equivalent. It still decides connection pooling:

- Serverless (Vercel): pooling is mandatory. Each invocation otherwise opens its own connection and exhausts max_connections under load; use a pooled endpoint or a proxy.
- Long-running Node (Railway, Render, VPS): a single mysql2 pool in the process is sufficient and simpler.

☐ Record the hosting target

☒ Record the decisions above, with rationale, in the repository — [DECISIONS.md](#)

3. Phase 1 — Database foundation

TIER 1 — blocks any real user

3.1 Credentials and connection

Never commit credentials

Put values in .env.local, which is already gitignored (.gitignore line 45: .env*).

Commit a .env.example with the keys present and values empty, so the required set is documented.

Rotate both database passwords if they have ever appeared in a chat, ticket, email, or shared document.

Required environment variables:

```

DATABASE_URL="mysql://parallax_app:pw@host:3306/parallax?ssl-mode=REQUIRED"
# same host and database, privileged user only — see 3.2
WRITER_DATABASE_URL="mysql://parallax_writer:pw@host:3306/parallax?..."
AUTH_SECRET="<openssl rand -base64 32>"
CHECKIN_TOKEN_SECRET="<openssl rand -hex 32>"
STORAGE_BUCKET_URL="..." # object storage for receipts
STORAGE_ACCESS_KEY="..."
STORAGE_SECRET_KEY="..."
SMTP_URL="..." # transactional email

```

- ☐ Create .env.local with production credentials (never committed)
- ☐ Create a separate .env for pre-production, pointing at the pre-prod database
- ☐ Commit .env.example listing every key with empty values
- ☐ Enforce ssl-mode=REQUIRED — use VERIFY_IDENTITY with the provider's CA certificate where available

- ☐ Confirm whether the database port is publicly reachable — a public MySQL port with password auth is a common breach vector
- ☐ IP-allowlist the application host if the provider supports it
- ☐ Configure connection pooling — mysql2 pool, or the provider's pooled endpoint on serverless
- ☒ Pin the session settings the schema depends on — [documented in MYSQL-MIGRATION.md](#)

Two connection settings are not optional

`timezone: 'Z'` — every timestamp column is DATETIME(3) holding UTC. Without this the driver silently shifts values by the server's offset.

`transaction_isolation = 'READ-COMMITTED'` — see 3.5. MySQL's REPEATABLE READ default makes capacity checks read stale data.

Also set `bigNumberStrings` (`xp_ledger.id` is BIGINT and JavaScript loses integer precision beyond 2^{53}) and `dateStrings` (the domain types are ISO strings, not Date objects).

3.2 Database users and privileges

Do not run the application as root.

MySQL privileges are cumulative and cannot be revoked at a narrower scope

Never write `GRANT ... ON parallax.*` and then try to REVOKE one table back — MySQL does not work that way, and the revoke silently leaves the database-level grant in force. Grant table by table instead.

```
-- Application user: cannot write the tables that grant value or hold money.
CREATE USER 'parallax_app'@'%' IDENTIFIED BY '<from secret manager>';
GRANT SELECT,INSERT,UPDATE,DELETE
      ON parallax.registrations TO 'parallax_app'@'%';
GRANT SELECT ON parallax.xp_ledger TO 'parallax_app'@'%';
GRANT SELECT ON parallax.attendance TO 'parallax_app'@'%';
GRANT SELECT ON parallax.user_roles TO 'parallax_app'@'%';
-- ...one line per table; full grant set in MYSQL-MIGRATION.md.
```

```
-- Privileged writer: awardXp, redeemCheckin, receipt review, role grants.
CREATE USER 'parallax_writer'@'%' IDENTIFIED BY '<from secret manager>';
GRANT INSERT      ON parallax.xp_ledger TO 'parallax_writer'@'%';
GRANT INSERT,UPDATE ON parallax.user_roles TO 'parallax_writer'@'%';
```

- ☐ Create the two least-privilege users above
- ☐ Grant table by table — never at database scope
- ☐ Create a separate read-only user if anyone needs analytics access
- ☐ Confirm the app cannot INSERT into `xp_ledger`, `attendance`, `user_roles` or `payment_receipts`
- ☒ Define the grant model that replaces RLS — [MYSQL-MIGRATION.md](#), “Authorization without RLS”

3.3 Character set, collation and enums

MySQL needs no extensions. What PostgreSQL got from pgcrypto and citext comes from the column definitions instead:

- `utf8mb4 / utf8mb4_0900_ai_ci` replaces `citext`. The default collation is already case-insensitive, so `UNIQUE` on `player_name` and email blocks impersonation by case variation for free. It is also accent-insensitive, which is a deliberate strengthening.
- `CHAR(36) CHARACTER SET ascii` replaces the `uuid` type — 36 bytes rather than the 144 a `utf8mb4 CHAR(36)` reserves. Ids are generated in TypeScript as `UUIDv7` for index locality; do not use MySQL's `UUID()`, which is v1 and encodes host MAC and time.
- `DATETIME(3)` replaces `timestampz`. Not `TIMESTAMP` — that type ends in 2038 and certificates outlive it.
- 14 per-column `ENUM`s replace the 12 named enum types. MySQL has no reusable `CREATE TYPE`, so adding a value is one `ALTER TABLE` per column — `MYSQL-MIGRATION.md` carries an inventory mapping each enum to the columns that use it, so that `ALTER` is findable.

☒ Set database charset and collation — `utf8mb4 / utf8mb4_0900_ai_ci`

☒ Define the 14 column enums and the inventory that tracks them — `MYSQL-MIGRATION.md`

☐ Add a CI check that each TypeScript union in `types.ts` still matches its column enum

3.4 Tables

All 27 tables are specified in `MYSQL-MIGRATION.md` and apply cleanly to MySQL 8.4: the 21 from the original spec, plus `payment_receipts`, `audit_log`, and the four `Auth.js` adapter tables.

Table group	Contents	Status
Original 21	<code>profiles</code> · <code>user_roles</code> · <code>colleges</code> · <code>departments</code> · <code>characters</code> · <code>levels</code> · <code>xp_ledger</code> · <code>event_categories</code> · <code>events</code> · <code>event_organizers</code> · <code>teams</code> · <code>team_members</code> · <code>registrations</code> · <code>attendance</code> · <code>checkin_token_redemptions</code> · <code>achievements</code> · <code>user_achievements</code> · <code>announcements</code> · <code>schedule_slots</code> · <code>sponsors</code> · <code>certificates</code>	Specified
<code>payment_receipts</code>	<code>storage_path</code> + <code>file_hash</code> , never file bytes. <code>unique(registration_id)</code> ; <code>file_hash</code> indexed so a receipt reused across accounts is detectable	Specified
<code>audit_log</code>	Append-only record of who did what. No FK on <code>actor_id</code> — deleting a user must not rewrite history	Specified
<code>users</code> / <code>accounts</code> / <code>sessions</code> / <code>verification_tokens</code>	<code>Auth.js</code> adapter tables, replacing Supabase <code>auth.users</code>	Specified
<code>announcement_reads</code> · <code>team_invites</code>	Per-user read state; invites beyond the join code	Optional — deferred

Resolved: the stale `skin_id` constraint

The previous spec's `CHECK` still listed copyrighted skin names

('steve','alex','creeper','enderman','miner'). Applied as written, every character insert would have failed.

It is now `ENUM('prospector','botanist','sentinel','voidwalker','artificer')`, matching `SkinId` in `src/backend/data/types.ts`, and there is a regression assertion for it in `scripts/check-schema.sh`.

Two latent defects that MySQL surfaced at DDL time

Both existed in the PostgreSQL spec too, where they would have failed at runtime instead:

`departments` — `UNIQUE(college_id, name)` never deduplicated generic departments, because `NULLs` are distinct in a unique key. Closed with a generated column. It must be `VIRTUAL`, not `STORED`: MySQL rejects `ON DELETE CASCADE` on a column a stored generated column derives from (errno 1215).

`payment_receipts` — `reviewed_by` carried `ON DELETE SET NULL` while also appearing in a `CHECK` that requires it to be non-null once decided. MySQL refuses the combination outright (errno 3823); the FK was dropped, since a reviewer id on a money record is history rather than a live reference.

- ☒ Adapt all `auth.users` references to the local `users` table — *FK types match at `CHAR(36)`; errno 3780 avoided*
- ☒ Write `payment_receipts` — `storage_path` + `file_hash`, not bytes
- ☒ Write `audit_log`
- ☒ Add the `Auth.js` adapter tables
- ☒ Correct the stale `skin_id` constraint — *verified by `scripts/check-schema.sh`*
- ☒ Remove the Supabase Realtime publication block — *`SUPABASE-MIGRATION.md` deleted; replaced by `MYSQL-MIGRATION.md`*
- ☐ Apply the DDL to the pre-production database as a versioned migration
- ☐ Write seed data — colleges, departments, 7 categories, 7 levels, ~15 achievements, sponsors
- ☐ Put migrations under version control and apply them via CI — never hand-run SQL against production

3.5 Integrity constraints — enforce in the database, not in application code

These invariants were enforced in JavaScript, which cannot survive concurrent requests. Seven are now plain keys; two need a transaction, because no constraint can express them.

Invariant	Mechanism
No double registration	<code>unique(event_id, user_id)</code> on registrations
No double check-in	<code>unique(event_id, user_id)</code> on attendance
Check-in token single-use	primary key(<code>jti</code>) on <code>checkin_token_redemptions</code>
XP grants idempotent	<code>unique(user_id, source_type, source_id, reason)</code> on <code>xp_ledger</code> — all four columns NOT NULL, or the key is useless for grants without a source
Achievement unlocks once	primary key(<code>user_id, achievement_id</code>)
One receipt per registration	<code>unique(registration_id)</code> on <code>payment_receipts</code>

Invariant	Mechanism
Player name unique	unique under utf8mb4_0900_ai_ci — blocks impersonation by case or accent variation
Capacity respected	SELECT ... FOR UPDATE on the event row, then a locking count. Never count-then-insert. Not GET_LOCK(), which is session-scoped and survives a rollback
Team size respected	SELECT ... FOR UPDATE on the team row, then a locking count, in the join transaction
total_xp cannot drift	Recompute as SUM() of the ledger, never increment

The trap: MySQL defaults to REPEATABLE READ, and PostgreSQL does not

Inside a transaction, a plain SELECT COUNT(*) is served from the snapshot taken at the transaction's first read. The capacity check can therefore be stale even while holding an exclusive lock on the event row — and capacity is exceeded.

Reproduced against the real schema: with session B inserting and committing a registration mid-transaction, session A's plain count returned 1 (stale) while the same count with FOR SHARE returned 2 (correct).

Two mitigations, both required: set transaction_isolation = 'READ-COMMITTED' on the application connection, AND make the count a locking read regardless, so correctness does not depend on the connection being configured properly.

- ☒ Specify all ten invariants and their mechanisms — [MYSQL-MIGRATION.md](#)
- ☒ Prove each one rejects what it claims to — [scripts/check-schema.sh](#) — 25 assertions, all passing
- ☐ Implement the two transactional invariants in the repository — capacity and team size
- ☐ Add deadlock retry on errno 1213 and lock-wait timeout on 1205

3.6 Transactions, views, indexes

The five PostgreSQL SECURITY DEFINER functions and five triggers are gone. Logic lives in TypeScript transactions, per the integrity decision in section 2.

Transactions (all four must reproduce the LocalRepository behaviour /dev/data-test already asserts):

- awardXp — ON DUPLICATE KEY UPDATE, branch on affectedRows, then recompute total_xp as SUM(). Not INSERT IGNORE, which also swallows FK violations and truncations
- registerForEvent — lock the event row, locking count, waitlist over capacity, XP only for a confirmed seat
- cancel — free the seat and promote the earliest waitlister. Lock order events → registrations, matching registerForEvent, or the two deadlock against each other
- redeemCheckin — jti insert and attendance insert in ONE transaction, so a replayed token fails even if the attendance row was later deleted

Triggers — only `updated_at` survives, and it needs no trigger: `DATETIME(3) ... ON UPDATE CURRENT_TIMESTAMP(3)` is native. Profile creation on signup, XP recompute, team-size enforcement and role-change events all move into the transactions above.

Views

- `leaderboard` — `RANK() OVER (ORDER BY total_xp DESC, created_at ASC)`. The tiebreak keeps ranks stable across reloads. `rank` is a reserved word and must be quoted
- `event_stats` — confirmed / waitlisted / checked-in counts and seats remaining.
`checked_in_count` is a scalar subquery because a MySQL view cannot contain a derived table

Indexes — nine, with the two PostgreSQL partial indexes converted to composites:

```
characters(total_xp DESC, created_at ASC) -- leaderboard (DESC honoured)
events(status, starts_at)
registrations(event_id, status)
registrations(user_id, registered_at DESC)
attendance(event_id, checked_in_at DESC)
xp_ledger(user_id, created_at DESC)
user_achievements(user_id, seen_at) -- was: WHERE seen_at IS NULL
announcements(published_at DESC)
payment_receipts(status, submitted_at) -- was: WHERE status='pending'
```

- ☒ Specify the four transactions, both views and all nine indexes — [MYSQL-MIGRATION.md](#)
- ☐ Implement the four transactions
- ☐ Create both views and all nine indexes in the target database

4. Phase 2 — Authentication

TIER 1 — blocks any real user

Unaffected by the database change, other than the adapter. One schema note: the Auth.js adapter's default `users.id` is `varchar(255)` and is overridden to `CHAR(36) ascii`, because every other table has a foreign key to it and MySQL requires FK column types to match exactly (errno 3780).

- ☐ Server-side password hashing with `bcrypt` or `argon2id` — the current browser SHA-256 is offline-attackable
- ☐ Sessions in `httpOnly + Secure + SameSite` cookies — `localStorage` is readable by any injected script
- ☐ Email verification before a registration counts — otherwise you cannot contact participants and anyone can create hundreds of fake accounts
- ☐ Password reset via single-use, time-limited, emailed token
- ☐ Real OAuth for Google / Discord / Microsoft — currently mocked; needs client IDs and secrets, a redirect-URI allowlist, and the state parameter for CSRF protection
- ☐ Failed-login throttling and account lockout
- ☐ Two-factor authentication for organizer and admin accounts — these accounts approve payments
- ☐ Session expiry and refresh policy
- ☐ Same-error response for unknown email and wrong password — avoids account enumeration

5. Phase 3 — Authorization

TIER 1 — blocks any real user

The mistake that gets applications like this compromised

Never store a role anywhere the client can write: not on the profile row, not in user metadata, not in a cookie the browser controls.

Role lives in a dedicated table whose write access is restricted to admins, and every mutation re-derives it from that table.

A role claim cached in a session token is acceptable for deciding what to render. It is never acceptable as the basis for a write.

MySQL has no row-level security — this is the real cost of the database decision

The previous design used RLS on `payment_receipts` and `user_roles` as a backstop for exactly the case where a developer forgets an authorization check. That backstop does not exist here and cannot be reconstructed in the schema.

Three partial compensations: table-level grants split across `parallax_app` and `parallax_writer`, so the ordinary connection physically cannot write the money and progression tables; an `assertRole()` call at the top of every mutating action; and an append-only `audit_log`.

Consequence to state plainly in code review: a missing authorization check on a mutating action is now unmitigated. Under RLS the database would have refused the write. It will not.

- ☐ Roles in a dedicated `user_roles` table, writable only by the privileged connection
- ☐ Every mutation re-derives role from the table, not from the session
- ☐ Per-event organizer scoping — the Photography Forest organizer must not read Hackathon Mine registrations
- ☐ Move route protection from the client guard into `middleware.ts` — the current guard is bypassable by disabling JavaScript
- ☐ `assertRole()` at the top of every mutating server action — no exceptions — there is no database-level backstop
- ☐ No client INSERT path for `xp_ledger` or `attendance` — enforced by grant, and awarded only by server transactions
- ☐ Route every privileged write through the `parallax_writer` connection, in a server-only module
- ☐ CI check that database credentials never appear in client-reachable code — `grep src/**` for `DATABASE_URL` and `WRITER_DATABASE_URL` outside `src/backend/db/`
- ☐ Make “is every mutating action role-asserted?” an explicit item on the review checklist

6. Phase 4 — Relocate data access to the server

TIER 1 — the largest single work item

The application is currently client-only. Because the database credentials cannot reach the browser, all 58 repository methods must execute on the server.

- ☐ Implement `MySQLRepository` against the existing `Repository` interface

- ☐ Expose it through Server Actions or Route Handlers only
- ☐ Convert read-only screens to Server Components
- ☐ Replace the client SessionProvider with a server-derived session
- ☐ Validate every input server-side with Zod — client-side validation is a convenience, not a control
- ☐ Return the existing DataError codes so current UI error handling still works
- ☐ /dev/data-test passes unchanged against MySQLRepository — this is the contract check

One known interface gap

PaymentReceipt.fileData is a base64 PDF string, because localStorage has no filesystem. The MySQL table deliberately stores storage_path and file_hash instead — a base64 PDF in a LONGTEXT column also collides with max_allowed_packet, commonly 4 MB on managed MySQL.

PaymentReceiptRepository is therefore the one place the repository seam genuinely leaks: submit() and the fileData field need reshaping when object storage lands, with /dev/data-test updated in the same commit.

7. Phase 5 — Payments

TIER 1 — highest risk area in the application

Recommendation: do not hand-verify PDF receipts

The current design — user uploads a PDF, an admin looks at it, an admin flips a status — is structurally fraud-prone. PDFs are trivially edited. One receipt can be submitted by several users. A screenshot of somebody else's payment passes visual review. Verification is a human judgement with no cryptographic backing, and it puts volunteers in the position of adjudicating money disputes.

Use a payment gateway instead (Razorpay is the standard in India; Stripe for international). The gateway confirms payment through a signed webhook to your server. Registration flips to paid only on a verified webhook. That is proof, not opinion.

7.1 If you use a payment gateway (recommended)

- ☐ Integrate hosted gateway checkout — never handle card data yourself — that puts you in PCI-DSS scope
- ☐ Verify webhook signatures on every callback
- ☐ Make webhook handling idempotent — gateways retry; a replayed webhook must not double-credit
- ☐ Reconcile gateway records against your own daily
- ☐ Record refunds and define a refund policy

7.2 If you keep manual receipt verification

All of the following are then required, not optional:

- ☐ Store receipts in private object storage — never a public bucket — they contain names, bank details, transaction IDs
- ☐ Serve them only via short-lived signed URLs
- ☐ Validate file type from magic bytes, not the extension

- ☐ Enforce a hard file-size cap and a page-count cap
- ☐ Never trust client-reported fileName or fileSizeBytes — both are client-supplied in the current type
- ☐ Sanitise filenames against path traversal
- ☒ Hash each file and flag reuse across users — *payment_receipts.file_hash is CHAR(64) and indexed*
- ☐ Malware-scan on upload
- ☐ Verification status writable only by admin, server-side — grant-enforced: only parallax_writer may UPDATE payment_receipts
- ☒ Require every decided receipt to name its reviewer and time — *CHECK constraint; asserted in scripts/check-schema.sh*
- ☐ Immutable audit entry for every verification decision — who, when, what note, from which address
- ☐ Stop storing PDFs as base64 in localStorage — will exceed the ~5 MB quota and start throwing

8. Phase 6 — File storage

TIER 2 — before fest day

Bucket	Access policy	Contents
receipts	Private. Signed short-lived URLs for admin review only.	Payment proof — contains personal and financial data
certificates	Private. Signed URLs to the owning participant.	Generated participation and winner certificates
gallery	Public read; insert gated on moderation.	Event photographs

- ☐ Provision three buckets with the policies above
- ☐ Type allowlist and size cap per bucket
- ☐ Confirm receipts bucket is not publicly listable

9. Phase 7 — QR check-in

TIER 2 — before fest day

A QR code containing a registration identifier is forgeable, infinitely replayable, and shareable — one screenshot in a group chat checks in forty people. Use short-lived signed tokens. The token design is database-agnostic and carried over unchanged.

- ☐ HMAC-signed rotating tokens, ~60 second validity — re-mint client-side every 25s; 60s survives bad venue wifi and scan queues
- ☐ Signing secret server-only — never in a NEXT_PUBLIC_ variable
- ☒ jti replay table so a token is single-use even inside its window — *checkin_token_redemptions; replay assertion passing*
- ☐ Verify endpoint asserts the scanner is an organizer of that event with scan permission
- ☐ Rate-limit the verify endpoint
- ☐ Serve over HTTPS — the camera API requires a secure context; test on a deployed preview, not localhost

- ☐ Audited manual fallback for dead phone batteries — method='manual' plus the organizer's id
- ☐ Allow ~5 seconds of clock skew
- ☐ Rehearse on the actual venue devices before the event

10. Phase 8 — Realtime and notifications

TIER 2 — before fest day

10.1 Realtime announcements

MySQL has no LISTEN/NOTIFY — announcements are polled

The previous design pushed rows over a subscription that respected row-level security, so scope filtering was automatic. Neither half of that exists here.

AnnouncementRepository.subscribe() keeps its signature, so no consumer code changes. It becomes a 15s query on published_at > lastSeen, served by the announcements(published_at DESC) index.

Scope filtering — global / event-registered / same-college — must now be an explicit server-side WHERE. Filtering in the client would mean every user receives every announcement and merely hides some of them.

- ☐ Implement the poll behind the existing subscribe() signature — 15s interval; stop when document.visibilityState is hidden
- ☐ Scope delivery server-side — global / event-registered / same-college — never filter in the client
- ☐ One poller per session, mounted in the authed layout — not per component
- ☐ Verify database connection limits against your plan — 2,000 simultaneous users polling will find them
- ☐ Refetch on reconnect — the poll is a nudge; the database is the truth

10.2 Transactional email

- ☐ Configure a provider — Resend, SES, or Postmark
- ☐ Publish SPF, DKIM and DMARC records — without these, verification mail lands in spam and users cannot sign up
- ☐ Handle bounces and complaints
- ☐ Unsubscribe link on anything non-transactional
- ☐ Templates: verification, password reset, registration confirmation, payment outcome, event reminder

11. Security hardening

11.1 Rate limiting and abuse control

Apply per-IP and per-user limits to each of these:

- Signup — prevents mass fake accounts
- Login — prevents credential stuffing
- Event registration — prevents seat scalping

- Check-in verification
- Receipt upload
- Announcement creation

- ☐ Implement rate limiting on all six endpoints
- ☐ Add a CAPTCHA on signup if abuse appears

11.2 Transport and headers

- ☐ HTTPS everywhere, with HSTS
- ☐ Content-Security-Policy
- ☐ X-Content-Type-Options: nosniff
- ☐ Frame-ancestors / X-Frame-Options
- ☐ Referrer-Policy
- ☐ CORS restricted to your own origins

11.3 Secrets management

- ☐ Verify .env files are gitignored — already true — .gitignore line 45
- ☐ Separate secrets for production and pre-production
- ☐ Document a rotation procedure and owner
- ☐ Scan git history for previously committed secrets

11.4 Application-level

- ☐ Parameterised queries only — Drizzle gives this by default; never interpolate SQL strings
- ☐ Server-side validation of every input
- ☐ Output encoding on anything user-generated — announcement bodies, team names, bios
- ☐ CSRF protection on state-changing requests
- ☐ Dependency vulnerability scanning in CI
- ☐ No stack traces or internal errors returned to clients

12. Legal and privacy

TIER 2 — statutory obligation, not paperwork

The application collects names, email addresses, phone numbers, college and department, year of study, and payment receipts from students. India's Digital Personal Data Protection Act 2023 applies.

- ☐ Publish a privacy notice at the point of collection
- ☐ Obtain explicit consent; apply purpose limitation
- ☐ Define and enforce a data retention policy
- ☐ Provide data export and deletion endpoints — user rights under the Act
- ☐ Document a breach notification procedure
- ☐ Name a grievance officer with contact details — an Act requirement

- ☐ Never log personal data — no emails or phone numbers in application logs
- ☐ Decide how to handle participants under 18 — first-year students can be 17; the Act requires verifiable parental consent for minors

One deletion detail the schema forces you to decide

`audit_log.actor_id` and `payment_receipts.reviewed_by` deliberately carry no foreign key, so deleting a user does not erase the record of what they did or which payments they approved. That is correct for an audit trail and for money.

It also means “delete my account” cannot be implemented as a plain row delete. Decide now what a deletion request removes and what it retains, and write it into the retention policy.

13. Operational readiness

TIER 3 — operational readiness

13.1 Backups and recovery

- ☐ Automated database backups
- ☐ Point-in-time recovery enabled — binary logging on, with a retention window
- ☐ Perform a restore drill — losing registrations mid-fest is unrecoverable if the restore has never been practised

13.2 Observability

- ☐ Error tracking — Sentry or equivalent
- ☐ Structured logs, scrubbed of personal data
- ☐ Uptime monitoring with alerting for fest day
- ☐ Slow-query monitoring — `slow_query_log` with a low `long_query_time` during the fest
- ☐ Audit log reviewable by an administrator

13.3 Load and capacity

The realistic worst case: registration opens and two thousand students hit it within five minutes, most targeting the same popular event.

- ☐ Load-test that specific spike
- ☐ Verify the capacity row lock holds under contention — and that the count inside the transaction is a locking read — a plain count silently over-admits under REPEATABLE READ
- ☐ Confirm deadlock retry works under contention — `errno 1213`; register and cancel contend on the same event row
- ☐ Confirm `max_connections` is not exhausted — especially on serverless, where each invocation may open its own connection
- ☐ Cache public event pages — they are read far more than they change

13.4 Environments

- ❑ Pre-production database in use and kept schema-identical to production
- ❑ Migrations applied to pre-production first, always
- ❑ Production access limited to named individuals
- ❑ Run scripts/check-schema.sh in CI — it fails the build if a schema change breaks an invariant

14. Go-live gates

Do not open registration until all five pass

These are verification gates, not tasks. Each one has caught a real class of defect in comparable systems.

- ❑ Authorization test script passes — exercise every mutating action as anonymous, player, organizer-of-A, organizer-of-B, and admin — asserting both allow AND deny. This matters more than it did under RLS: the application layer is now the only layer
- ❑ /dev/data-test passes unchanged against MySQLRepository — 40 assertions covering idempotent XP, no double registration, capacity, waitlist promotion
- ❑ scripts/check-schema.sh passes against the production schema — proves the constraints in the live database, not just in the migration file
- ❑ Adversarial review of the payment flow specifically
- ❑ Load test at fest-day concurrency passes — including the deadlock-retry path
- ❑ Backup restore drill completed successfully

15. Appendix — reference

15.1 Repository surface to implement

13 sub-repositories, 58 methods total:

auth (7)	signUp signIn signInWithProvider signOut getSession onAuthStateChange grantRole
profiles (2)	get update
characters (5)	getByUser getByPlayerName create update isPlayerNameTaken
events (5)	list getBySlug getById stats schedule
registrations (5)	listForUser listForEvent get register cancel
teams (7)	listForUser getById getByJoinCode members create join leave
attendance (3)	listForUser listForEvent checkIn
achievements (6)	listAll listForUser unlock listUnseen markSeen evaluate
xp (2)	award ledger
leaderboard (2)	top rankOf
announcements (3)	list create subscribe
reference (5)	colleges departments categories levels sponsors
paymentReceipts (6)	submit getByRegistration getByUser listForEvent listPending review

15.2 Error codes the UI already handles

NOT_AUTHENTICATED INVALID_CREDENTIALS EMAIL_TAKEN

PLAYER_NAME_TAKEN	NOT_FOUND	ALREADY_REGISTERED
EVENT_FULL	REGISTRATION_CLOSED	TEAM_FULL
INVALID_JOIN_CODE	STORAGE_UNAVAILABLE	VALIDATION_FAILED
RECEIPT_ALREADY_SUBMITTED		PAYMENT_NOT_VERIFIED

The server should return these same codes so existing error handling continues to work.

15.3 Existing documents in the repository

File	Contents
MYSQL-MIGRATION.md	Schema of record. Full MySQL 8.4 DDL for 27 tables, translation rules from the PostgreSQL design, the REPEATABLE READ trap, the four transactions, both views, indexes, and the grant model that replaces RLS
DECISIONS.md	The five locked decisions with rationale and cost; supersedes this document on database choice; lists what the previous PostgreSQL revision got wrong
scripts/check-schema.sh	Applies the DDL from MYSQL-MIGRATION.md to a scratch database and asserts every invariant rejects what it claims to. 25 assertions. Run it in CI
ANIMATION.md	Animation rules and known pitfalls
ART-ASSETS.md	Asset specifications and the originality requirement
VOXEL-3D.md	3D village architecture and performance findings
/dev/data-test	40-assertion acceptance suite — the backend contract check

End of document